

UrbanCode Edutech — Complete Production Audit Report

Project: Urbancode Next.js (uc-next)
Repository: d:\urbancode\Urbancode-nextJs
Audit Date: 27 August 2026
Audit Type: Read-only (no code modified)
Scope: Full codebase — frontend, API routes, MongoDB, compiler backend, deployment, security, performance, UX, testing, architecture

Document Control

Field	Value
Prepared for	UrbanCode Edutech Engineering Team
Methodology	Static code review, route/API inventory, dependency analysis, configuration review
Auditors (roles simulated)	Software Architect, Staff Engineer, QA, Security, Performance, DevOps, Product
Classification	Internal — contains security findings (credentials redacted)

Table of Contents

- [1. Executive Summary](#)
- [2. Complete Project Audit](#)
- [3. Production Folder Structure & Architecture](#)
- [4. Bug & Incomplete Work Audit](#)
- [5. Performance & Speed Optimization](#)
- [6. Security Audit](#)
- [7. Database & API Audit](#)
- [8. Testing & Production Readiness](#)
- [9. UX, Features & 200% Improvement](#)
- [10. Code Quality & AI-Generated Code Audit](#)
- [11. Final Master Task List & Roadmap](#)
- [12. Appendix — Health Scores](#)

1. Executive Summary

What This Application Is

UrbanCode Edutech is a Next.js 15 marketing and education platform for an IT training institute in Chennai (and Tirunelveli). It serves:

- **Prospective students** — browse courses, enroll, book demos, contact, study abroad info
- **Kids programs** — kidz-space courses and playzone
- **Students** — online code compiler, quizzes, portfolio showcase
- **Staff/admins** — feedback collection dashboard, compiler problem admin (limited)
- **Marketing/SEO** — blogs, RSS (LinkedIn), Google Reviews, structured data

Technology Stack

Layer	Technology
Frontend	Next.js 15.5 (App Router), React 19, Bootstrap 5, Framer Motion, GSAP
API	Next.js Route Handlers (app/api/**)
Database	MongoDB via Mongoose (feedback DB, Google reviews DB)
Secondary backend	Express on Render (compiler/compiler/backend) — compiler + duplicate feedback
Email	Gmail SMTP (Nodemailer)
CRM	Zen CRM (api.zen-urbancode.in)
AI	Google Gemini (course chatbot)
Media	Cloudinary (video URLs), large static assets in public/
Deployment	Vercel (Next.js), Render (compiler backend), weekly cron for reviews sync

Overall Project Health Score: 42 / 100

The site is **live and functional for marketing and lead capture**, but it is **not production-grade** by enterprise standards. Critical security issues (hardcoded credentials, unprotected admin APIs), zero automated tests, no CI/CD, and significant architectural drift from AI-assisted development block calling this a hardened production system.

Top 5 Problems Another Engineering Team Would Complain About First

1. **Secrets and credentials embedded in source code** with environment fallbacks (MongoDB URLs, admin passwords, JWT/cron secrets, compiler admin login in client JS).
2. **Feedback admin APIs have no server-side authentication** — JWT is cosmetic; anyone can read/delete feedback data.
3. **No tests, no CI/CD pipeline** — every deploy is a manual gamble.
4. **~10,000-line monolithic data files and duplicated codebases** (`coursesData.js` , `StudyAbroad.css` , vendored portfolio) make changes risky and bundles heavy.
5. **Inconsistent SEO/canonical URLs** (www vs non-www), incomplete sitemaps, and missing metadata on high-traffic routes hurt discoverability and trust.

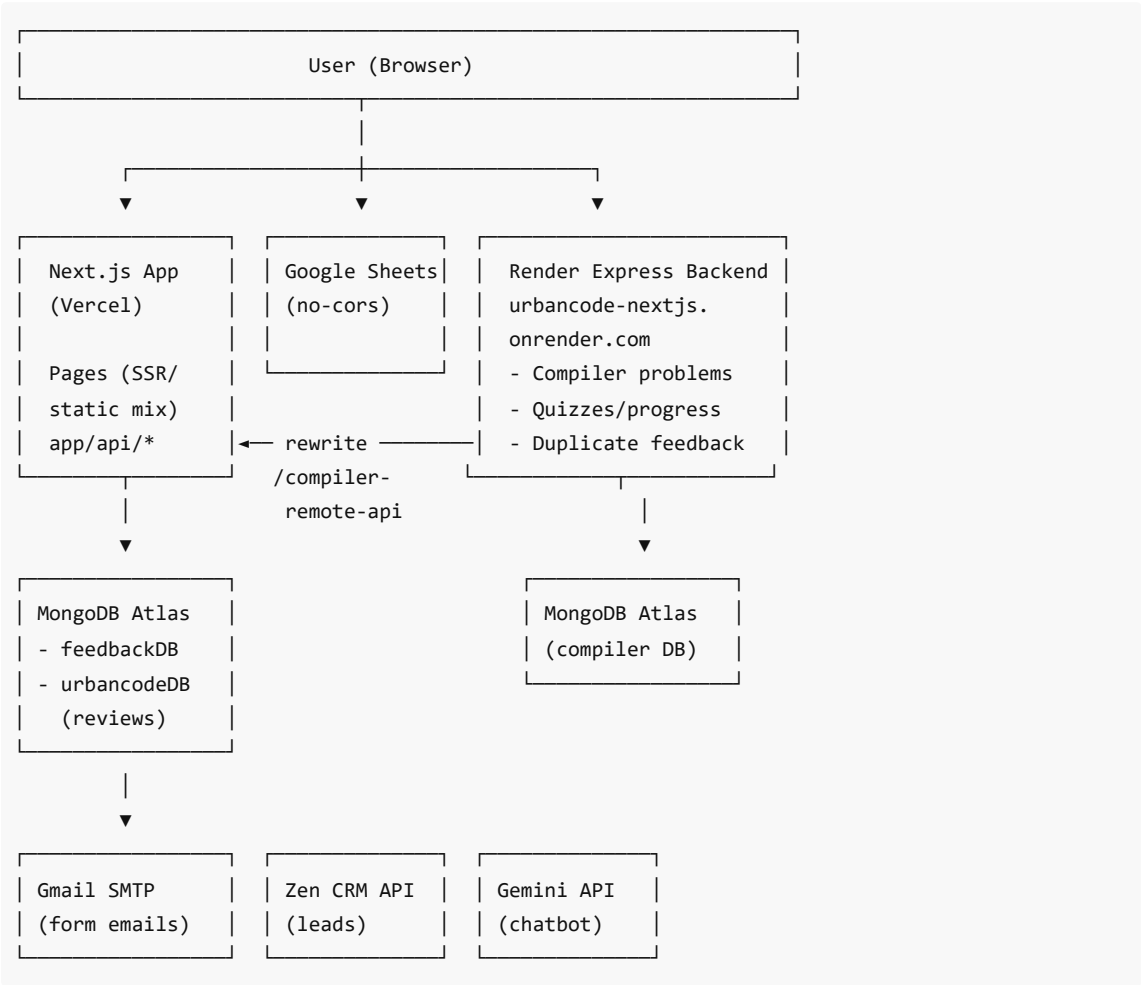
Score Summary

Dimension	Score (/100)
Functionality	68
Security	22
Performance	45
Architecture	38
UX	58
Testing	5
DevOps	35

Scalability	40
Maintainability	32
Overall Production Readiness	42

2. Complete Project Audit

2.1 System Architecture Understanding



2.2 Major Modules

Module	Route(s)	Status	Notes
Homepage	/	✅ Working	Heavy client components, dynamic imports for below-fold
Courses catalog	/courses, /courses/[category], /courses/[cat]/[course]	⚠️ Partial	Wrong category fallback bug; missing categories in static params
Blogs	/blogs, /blogs/[slug], /rss.xml	✅ Working	RSS fixed recently; metadata on detail only

Study Abroad	/study-abroad, country pages	⚠️ Partial	No metadata; 3s nav delay; Dubai link broken
Contact / Forms	/contact-us, /form, /book-demo	⚠️ Partial	/form shows success without API check
Compiler	/compiler	⚠️ Partial	HashRouter; client-only admin auth
Portfolio	/portfolio	⚠️ Partial	Catch-all routes render same page
Feedback system	/feedback, /feedback/admin/*	🔴 At risk	APIs unprotected server-side
Kids courses	/kids-courses, /playzone	✅ Working	—
Certifications	/certifications/[slug]	⚠️ Partial	No metadata found
Email/API layer	app/api/send-email/*	⚠️ Partial	No rate limiting; spam checks removed on most routes
Google Reviews	/api/google-reviews	✅ Working	Depends on MongoDB + cron sync

2.3 User Workflows

Course enrollment: Home/course CTA → Enquiry modal or /form → /api/send-email/course-enquiry → Gmail + optional CRM enrollment.

Contact: /contact-us → /api/send-email/contact → Gmail (+ CRM if course enquiry interest).

Book demo: Widget or /book-demo → email API + Google Sheets (no-cors, fire-and-forget).

Study abroad: Nav (3s animation) → hub page → separate form routes; CRM skipped by design.

Feedback: Public form → POST /api/feedback/responses . Admin views data via pages that store JWT in localStorage — **APIs do not verify token.**

Compiler: UI loads from Next; data from Render backend or local fallback JSON.

2.4 Confirmed Production Risks

Risk	Severity	Evidence
Credential leakage in git history	Critical	Hardcoded MongoDB URIs in lib/feedbackDb.js, lib/googleReviewsDb.js, migration scripts
Broken access control on feedback APIs	Critical	No JWT verification in app/api/feedback/*
Public API abuse (email spam, Gemini cost)	High	No rate limiting on /api/send-email/*, /api/chat
PII exposure via GET feedback endpoints	High	GET /api/feedback/responses unauthenticated

Form false-success UX	High	app/form/page.jsx ignores API failure
SEO split (www/non-www)	Medium	Mixed canonicals across pages
Bundle bloat from coursesData.js on client	Medium	~396KB+ imported in client components
Static asset weight (~837MB in public/)	Medium	Large videos/images served raw

2.5 Potential Risks (Unconfirmed Without Runtime Testing)

- CRM API downtime silently falling back to static course list without admin alert
- Render cold-start latency affecting compiler UX
- MongoDB connection pool exhaustion under concurrent feedback submissions
- Gmail SMTP rate limits during marketing campaigns
- LinkedIn RSS sync delays (external platform behavior)

2.6 Technical Debt Highlights

- Duplicate portfolio codebases (app/portfolio/ vs app/_projects/UC_Portfolio-main/)
- Duplicate feedback implementations (Next.js API vs Express backend)
- Root-level scratch files (temp_data.js , keys.txt , migration scripts)
- Dead dependencies (@emailjs/browser , unused lenis smooth scroll)
- Commented-out homepage sections and entire hero components
- Incomplete .env.example vs actual env surface area

3. Production Folder Structure & Architecture

3.1 Current Structure (Simplified)

Urbancode-nextJs/	
└─ app/	# Next.js App Router (pages, API, components mixed)
├─ api/	# Route handlers
├─ components/	# Shared UI (large, flat-ish)
├─ courses/	# Course pages + 10k-line coursesData.js
├─ compiler/	# Embedded SPA-style compiler
├─ portfolio/	# Portfolio SPA
├─ _projects/	# NON-ROUTABLE legacy portfolio clone
├─ study-abroad/	# Large client pages + 6k-line CSS
└─ ...	
└─ compiler/compiler/backend/	# Separate Express app
└─ lib/	# DB, mail, CRM helpers (good start)
└─ public/	# ~837MB static assets
└─ components/	# Orphan KidsLoader only
└─ temp_data.js	# 7k lines – should not be in repo root
└─ docs/	

3.2 Architectural Problems

Problem	Example
---------	---------

Business logic in UI	Course slug mapping, CRM logic scattered in forms
Data layer in UI folder	app/courses/[categorySlug]/coursesData.js (10,244 lines)
Components inside app/	app/components/ — valid for Next but lacks domain separation
Private folder misuse	app/_projects/ contains routable-looking page but _ makes it private
Dual backends	Next API + Express on Render for overlapping concerns
No src/ or domain modules	Everything at repo root

3.3 Recommended Production Folder Structure

Architecture tailored to **this** app (content-heavy marketing site + selective dynamic features):

```

Urbancode-nextJs/
├─ app/                                # ROUTING ONLY – thin page shells
│  ├─ (marketing)/                     # Route group: public pages
│  │  ├─ page.js
│  │  ├─ courses/...
│  │  ├─ blogs/...
│  │  └─ study-abroad/...
│  ├─ (tools)/                         # compiler, portfolio
│  ├─ api/                             # Route handlers – delegate to services
│  ├─ layout.js
│  ├─ not-found.js
│  ├─ error.js
│  └─ loading.js
├─ src/
│  ├─ components/                     # Presentational UI
│  │  ├─ layout/
│  │  ├─ home/
│  │  ├─ courses/
│  │  ├─ blog/
│  │  └─ forms/
│  ├─ features/                       # Feature modules (logic + hooks)
│  │  ├─ courses/
│  │  ├─ enquiry/
│  │  ├─ feedback/
│  │  ├─ compiler/
│  │  └─ study-abroad/
│  ├─ services/                      # Server-side business logic
│  │  ├─ email/
│  │  ├─ crm/
│  │  ├─ feedback/
│  │  └─ reviews/
│  ├─ lib/                           # Infra: db, auth, validators
│  │  ├─ db/
│  │  ├─ auth/
│  │  └─ validators/
│  └─ data/                          # Static JSON/content (split by domain)

```

```

| | | └─ courses/
| | | └─ blogs/
| | | └─ study-abroad/
| | └─ hooks/
| | └─ schemas/           # Zod schemas (already exists partially)
| | └─ types/
|
| └─ compiler-backend/    # Rename from compiler/compiler/backend
| └─ public/              # Optimized assets only
| └─ scripts/             # One-off migrations (not in root)
| └─ tests/
|   └─ unit/
|   └─ integration/
|   └─ e2e/
└─ docs/

```

3.4 File Actions (Move / Merge / Remove)

Action	Target	Reason
Remove	temp_data.js, keys.txt, root migration scripts from repo	Scratch/credential risk
Remove	app/_projects/UC_Portfolio-main/	Duplicate of app/portfolio/
Remove	app/components/Home/Faqs copy.jsx	Duplicate with wrong content
Split	coursesData.js → src/data/courses/*.json per category	Maintainability
Split	StudyAbroad.css → modular CSS modules	6,775 lines duplicated
Move	app/components/* → src/components/*	Consistent imports
Move	Email logic from routes → src/services/email/	Thin controllers
Merge	Feedback API (pick Next OR Express, not both)	Single source of truth
Add	middleware.ts	Auth, redirects, security headers
Add	app/not-found.js, error.js, loading.js	Production UX

3.5 Safe Restructuring Plan (Phased)

Phase 1 — No moves (1 week): Add middleware, not-found, error boundaries, env validation; remove secrets from source.

Phase 2 — Data extraction (2 weeks): Split `coursesData.js` and `blogsData.json` imports through a single `@/data` index; delete `temp_data.js`.

Phase 3 — Component migration (2–3 weeks): Move `app/components` → `src/components` with path alias update.

Phase 4 — Service layer (2 weeks): Extract API route bodies into `src/services/*`.

Phase 5 — Backend consolidation (2 weeks): Deprecate duplicate Express feedback routes or proxy exclusively through Next.

4. Bug & Incomplete Work Audit

4.1 Confirmed Bugs (P0–P1)

ID	Module	File	Problem	Root Cause	Impact
B01	Security	lib/feedbackDb.js, lib/googleReviewsDb.js	MongoDB credentials in source	Env fallback hardcoded	Full DB compromise if repo leaked
B02	Security	app/api/feedback/*	Admin CRUD without auth	JWT never verified server-side	Data breach, data deletion
B03	Security	app/compiler/.../AdminDashboard.jsx	Admin password in client JS	Client-side auth only	Compiler admin takeover
B04	Forms	app/form/page.jsx	Success shown on API failure	No response.ok check	Lost leads, false user confidence
B05	Courses	app/courses/[categorySlug]/Courses.jsx	Wrong category for unknown slug	deslugify() defaults to Fullstack	Wrong courses shown for Health Care, App Dev
B06	Routing	app/training/[slug]/page.jsx	Invalid slug renders broken page	No notFound()	404 UX / SEO junk
B07	React	app/study-abroad/page.jsx	Hooks inside .map()	Rules of Hooks violation	Unpredictable runtime bugs
B08	React	app/components/ChatbotWidget.jsx	Stale pathname in effect	Missing dependency	Widget visibility wrong after nav
B09	SEO	Multiple files	www vs non-www split	No canonical redirect	Duplicate content penalty
B10	Nav	app/components/header/Navbar.jsx	3s forced delay on Study Abroad click	Intentional animation gate	High bounce/friction

B11	Links	app/components/OneOnOneWidget.jsx	Dubai study link with no page	Missing route	404 for users
B12	Portfolio	app/portfolio/page.jsx	Sub-routes all render HomePage	Catch-all no-op	Broken deep links
B13	Email	app/api/send-email/send-curriculum/route.js	SSRF via brochureUrl	Unrestricted fetch	Internal network probe risk
B14	Email	Multiple send-email routes	HTML injection in email body	Unescaped user input	Phishing via outbound emails

4.2 Incomplete Work List

ID	Feature	Location	Status
I01	/projects route	Referenced in sitemap; page in _projects (private)	Broken/unreachable
I02	Dubai study abroad page	Widget link exists	Not implemented
I03	Homepage Coming Soon popup	app/page.js commented out	Disabled
I04	GetCertified section	app/page.js commented out	Disabled
I05	Cinematic contact loader	contactUs.jsx — loader never triggered	Dead feature
I06	Course category metadata	[categorySlug]/page.jsx	Missing generateMetadata
I07	Blogs listing SEO	blogs/page.js (client)	No metadata
I08	Study abroad SEO	All study-abroad pages	No metadata
I09	Health Care / App Development categories	In data, not in nav/static params	Partially unreachable
I10	Events "COMING SOON 2026"	Portfolio events data	Placeholder content
I11	Trending courses "Poster Coming Soon"	TrendingCourses.jsx	Placeholder labels
I12	Compiler problem admin protection	Express backend comments	Not implemented
I13	Spam validation on email routes	Comment "Spam Validation removed"	Regressed

4.3 Production Blockers

1. Rotate and remove all hardcoded credentials from repository (and git history if ever committed).
2. Implement server-side JWT middleware for feedback admin APIs.
3. Add rate limiting to public POST endpoints (`/api/send-email/*` , `/api/chat` , `/api/feedback/responses`).
4. Fix form submission success/failure handling on `/form` .
5. Add automated smoke tests for critical lead-capture flows before major marketing pushes.

5. Performance & Speed Optimization

5.1 Frontend Performance Issues

Issue	Why It's Slow	Recommendation	Impact	Priority	Complexity
<code>images.unoptimized: true</code>	No resize/WebP/srcset	Enable Next image optimization on Vercel	High LCP improvement	P0	S
~837MB <code>public/assets</code>	Raw MP4/JPG/PNG downloads	Compress, Cloudinary, lazy video	Massive bandwidth savings	P0	L
<code>coursesData.js</code> in client bundle	~396KB JSON parsed in browser	Server-fetch or split by category	Faster course pages	P1	M
Navbar loads 3 modals globally	Framer Motion + Swal + forms on every page	Dynamic import widgets	Smaller initial JS	P1	M
Bootstrap + FA CDN globally	CSS/JS on all routes	Tree-shake; load icons per-page	Faster first paint	P2	M
Study abroad 1,459-line client page	Large JS + 5 CSS files	Split routes; RSC where possible	Faster TTI	P2	L
Duplicate <code>/api/courses</code> fetches	Each form/modal fetches independently	Shared React context or SWR cache	Fewer network calls	P1	M
No route <code>loading.js</code>	Blank waits on navigation	Add skeleton loading states	Perceived speed	P2	S

5.2 Backend / API Performance

Issue	Recommendation	Impact
<code>/api/google-reviews</code> no cache	Add <code>revalidate: 3600</code> or edge cache	Reduce Mongo reads
CRM resolve uses no-store	Cache course name map 1 hour	Faster email pipeline

No compression headers documented	Ensure Vercel gzip/brotli (default)	Smaller payloads
-----------------------------------	-------------------------------------	------------------

5.3 Database Performance

Issue	Recommendation
Feedback schemas strict: false	Define indexes on createdAt, trainerId
No pagination on GET responses	Paginate admin list endpoints
Potential N+1 on analytics route	Aggregate pipeline review needed at scale

5.4 Scalability Outlook

Scale	Expected Pain Points
10× traffic	Email/Gemini API abuse; Mongo connection limits
100× traffic	coursesData bundle; unbounded GET feedback lists
1000× traffic	Requires CDN for assets, Redis rate limit, read replicas, background email queue

5.5 Top 10 Performance Improvements (Biggest Real-World Gains)

1. **Enable Next.js image optimization** — remove `unoptimized: true` where hosting supports it.
2. **Compress and offload videos** to Cloudinary (already partially integrated).
3. **Split `coursesData.js`** — load category data server-side only.
4. **Lazy-load navbar widgets** (Chatbot, BookDemo, OneOnOne).
5. **Deduplicate `/api/courses` fetches** with shared client cache.
6. **Add caching to Google Reviews API** route.
7. **Convert large JPG course images to WebP** in `public/images/courses/`.
8. **Dynamic import study-abroad page** or split by country route (already separate — add metadata + code split).
9. **Remove dead CSS** from global layout (`homeCarouselNav.css` on non-home pages).
10. **Remove 3-second Study Abroad nav delay** — optional skip for returning users.

6. Security Audit

6.1 Security Score: 22 / 100

6.2 Findings by Severity

Critical

ID	Category	Location	Problem	Remediation
S01	Secrets	lib/feedbackDb.js, lib/googleReviewsDb.js, backend configs	MongoDB URI fallbacks in source	Remove fallbacks; rotate credentials; use Vercel env only
S02	Auth	app/api/auth/login/route.js	Default admin username/password in code	Env-only; strong passwords; rotate

S03	Auth	All app/api/feedback/* mutating/list routes	No JWT verification	Add verifyAdminToken() middleware
S04	Secrets	vercel.json + sync route	Cron secret in URL query string	Use Authorization header only; rotate secret

High

ID	Category	Location	Problem	Remediation
S05	Auth	AdminDashboard.jsx	Compiler admin creds in client	Server-side auth; env-based
S06	Abuse	/api/chat	Public Gemini proxy, no rate limit	Auth or IP rate limit; cap tokens
S07	Abuse	/api/send-email/*	No rate limiting	Upstash/Vercel KV rate limiter
S08	Exposure	GET /api/feedback/responses	Full PII dataset public	Require admin auth
S09	SSRF	send-curriculum/route.js	Fetches user-supplied URL	Allowlist /public/curriculum/* paths only

Medium

ID	Category	Problem	Remediation
S10	XSS	HTML email templates unescaped	Use escapeHtml everywhere (like ielts route)
S11	Headers	No CSP, HSTS, X-Frame-Options	Add headers() in next.config or middleware
S12	CSRF	No tokens on POST APIs	SameSite cookies if session added; rate limit
S13	CORS	Compiler backend allows missing Origin	Tighten production CORS
S14	Client	Google Apps Script URL in client	Move to server proxy if sensitive

Low

ID	Problem	Remediation
S15	Admin UI auth = localStorage only	HttpOnly cookie session
S16	.env.example incomplete	Document all required vars
S17	robots.js VS public/robots.txt conflict	Single source

6.3 Security Hardening Roadmap

Week 1 (Emergency): Rotate all exposed credentials; remove hardcoded fallbacks; block unauthenticated feedback admin APIs.

Week 2: Rate limiting on email + chat; security headers; fix SSRF and HTML injection.

Week 3: Middleware for `/feedback/admin` routes; server-side compiler admin; audit git history for leaked secrets.

Week 4: Dependency audit (`npm audit`); pen-test lead forms; document security runbook.

7. Database & API Audit

7.1 Database Assessment

Technology: MongoDB Atlas via Mongoose (no SQL, no Prisma).

Database	Purpose	Connection File
feedbackDB	Feedback questions, trainers, responses	lib/feedbackDb.js
urbancodeDB	Google Reviews cache	lib/googleReviewsDb.js
Compiler DB	Problems, quizzes, student progress	compiler/.../config/db.js

Schema concerns:

- `strict: false` on feedback models — arbitrary document injection
- No documented indexes for analytics queries
- No soft-delete or audit fields on feedback responses
- No migration framework (ad-hoc scripts at repo root)

Recommendations:

- Define explicit Mongoose schemas with validation.
- Add indexes: `{ createdAt: -1 }`, `{ trainerId: 1 }`, `{ isActive: 1 }`.
- Paginate all list endpoints (default `limit=50`).
- Consolidate to one MongoDB cluster/database strategy document.
- Remove migration scripts from production repo; use proper migration tool.

7.2 API Assessment

22 Next.js API routes + Express backend proxy.

API Group	REST Consistency	Auth	Validation	Caching
<code>/api/send-email/*</code>	POST only	None	Basic regex	None
<code>/api/feedback/*</code>	Mixed	None	Minimal	None
<code>/api/courses</code>	GET	Server key	N/A	1hr revalidate
<code>/api/google-reviews</code>	GET	None	N/A	None
<code>/api/chat</code>	POST	None	None	None
<code>/api/auth/login</code>	POST	N/A	Weak	None

API improvements prioritized:

- Standard error response shape `{ error, code, message }`.
- Zod validation on all POST bodies.
- Admin auth middleware on sensitive routes.

- 4. Pagination on GET list endpoints.
- 5. API versioning prefix (/api/v1/) before external consumers depend on shapes.
- 6. Idempotency keys for enrollment submissions (prevent duplicate CRM leads).

7.3 Scalability Risks at Higher Data Volume

- Unbounded GET /api/feedback/responses will OOM or timeout with thousands of records.
- Google Reviews sync without deduplication strategy may grow unbounded (verify unique index on review ID).
- Compiler progress tracking on shared Render instance — single point of failure.

8. Testing & Production Readiness

8.1 Production Readiness Score: 38 / 100

8.2 Testing Status

Test Type	Status	Gap
Unit tests	✗ None	0% coverage
Integration tests	✗ None	API routes untested
E2E tests	✗ None	Enrollment flow untested
Auth tests	✗ None	Feedback admin bypass undetected
Performance tests	✗ None	No Lighthouse CI
Regression tests	✗ None	Manual only

Critical workflows needing tests first:

1. Course enquiry → email + CRM enrollment
2. Contact form → email delivery
3. Feedback submission → MongoDB persist
4. Feedback admin login → authorized CRUD only
5. Google Reviews sync cron
6. RSS feed generation with new blog posts
7. Compiler problem fetch (remote + fallback)

8.3 DevOps & Infrastructure

Item	Status	Notes
CI/CD (GitHub Actions)	✗ Missing	No automated build/test/deploy
Docker	✗ Missing	Render deploys Node directly
Environment variables	⚠ Partial	.env.example incomplete
Secrets management	● Poor	Hardcoded fallbacks in code
SSL/HTTPS	✓ Vercel default	—
Cron jobs	✓ vercel.json weekly reviews sync	Secret in URL — fix

Health checks	❌ Missing	No <code>/api/health</code> endpoint
Logging	⚠️ Basic	<code>console.error</code> only
Monitoring/alerting	❌ Missing	No Sentry/Datadog
Error tracking	❌ Missing	—
Backup strategy	❌ Not documented	MongoDB Atlas backups assumed
Rollback strategy	⚠️ Vercel instant rollback	Not documented
Database migrations	❌ Ad-hoc scripts	Risky

8.4 Failure Mode Behavior

Failure	Current Behavior	Risk
MongoDB down	API 500; reviews/feedback break	No graceful degradation page
Gmail SMTP failure	500 with auth hint	User sees error on some forms; <code>/form</code> may still show success
CRM API down	Fallback static courses; enrollment may fail silently	Lost leads
Render backend cold start	Compiler slow/timeout	Poor student UX
Gemini quota exceeded	Chat returns fallback message	OK-ish

8.5 Production Readiness Checklist

- ☐ Remove all secrets from source code
- ☐ Server-side admin authentication
- ☐ Rate limiting on public APIs
- ☐ Fix form false-success bug
- ☐ Add `not-found.js`, `error.js`, `loading.js`
- ☐ Unified `www` canonical redirect
- ☐ Complete sitemap (blogs, study-abroad, courses)
- ☐ Automated test suite (minimum smoke tests)
- ☐ CI pipeline (lint + test + build on PR)
- ☐ Sentry or equivalent error tracking
- ☐ `/api/health` endpoint
- ☐ Documented env vars and deployment runbook
- ☐ Asset optimization (images/videos)
- ☐ Remove duplicate codebases and temp files

9. UX, Features & 200% Improvement

9.1 User Journey Assessment

Journey	UX Quality	Key Issues
Discover course	Good visuals	Too many CTAs; category bug for some slugs
Enroll / enquire	Functional	False success on /form; duplicate modals
Book demo	Functional	No SEO; dual submit paths opaque to user
Contact	Polished UI	Dead cinematic loader; course fetch on every visit
Study abroad	Rich content	3s nav delay; fragmented forms; no Dubai page
Read blog	Clean	Missing listing metadata; FAQ accordion good
Use compiler	Feature-rich	Hash URLs; admin insecure; cold start
Admin feedback	Basic dashboard	Security undermines trust

9.2 Highest-Value UX Improvements

- 1. **Remove 3-second Study Abroad navigation delay** (or show skip).
- 2. **Single primary CTA strategy** — one clear "Enquire Now" path vs 5 floating widgets.
- 3. **Fix form success/error feedback** on all submission pages.
- 4. **Add skeleton loaders** on course and blog pages.
- 5. **Consolidate floating widgets** into one expandable FAB on mobile.
- 6. **Accessible navigation** — hamburger aria-expanded , keyboard focus traps in modals.
- 7. **Blog listing page metadata** for SEO traffic to /blogs .
- 8. **Course category page titles/descriptions** for organic search.

9.3 Missing Features (High Business Value)

Feature	User/Business Problem Solved
Lead dashboard	Track enquiry → CRM → conversion status
WhatsApp follow-up automation	Faster response to hot leads
Course comparison tool	Help undecided students choose
Batch schedule / seat availability	Reduce "when does it start?" calls
Student portal login	Progress, certificates, batch info
Placement stats dashboard	Social proof with live data
IELTS mock test booking	Monetize proficiency vertical
Blog search + categories filter	Content discoverability
Admin CMS for blogs/courses	Stop editing 10k-line JSON files
Analytics dashboard	Which courses/pages convert

9.4 AI & Automation Opportunities

Opportunity	Description
Expand Gemini chatbot	Course recommendation → direct enquiry pre-fill
AI blog SEO assistant	Meta tags, internal linking suggestions (partially manual today)
Lead scoring	Classify enquiry quality from form text
Automated review responses	Draft replies to Google reviews for admin approval
Curriculum Q&A bot	RAG over PDF curricula already in <code>public/curriculum/</code>
Form spam AI filter	Replace removed regex spam checks

9.5 "200% Better" Strategic Themes

- 1. **Trust & security** — fix admin APIs; display security-conscious brand.
- 2. **Speed** — half LCP via images/video optimization.
- 3. **Conversion** — one funnel, reliable submissions, CRM visibility.
- 4. **Content ops** — CMS instead of JSON surgery for blogs/courses.
- 5. **Student retention** — portal + compiler progress tied to batches.
- 6. **International** — complete study-abroad country pages + CRM pipeline.
- 7. **Observability** — know when leads fail before customers complain.

10. Code Quality & AI-Generated Code Audit

10.1 Maintainability Score: 32 / 100

10.2 AI-Agent Inconsistency Patterns Observed

Pattern	Evidence
Duplicate implementations	Portfolio ×2, feedback ×2, blog data import paths ×2
Mixed URL conventions	<code>urbancode.in</code> VS <code>www.urbancode.in</code>
Mixed auth approaches	JWT <code>localStorage</code> vs <code>sessionStorage</code> vs hardcoded
Mixed form patterns	Some use Zod+honeypot; others stripped spam checks
Mixed routing	Next App Router + react-router HashRouter in <code>compiler/portfolio</code>
Copy-paste CSS	<code>StudyAbroad.css</code> duplicated sections
Dead commented blocks	Entire hero components commented out
Oversized generated files	<code>coursesData.js</code> , <code>temp_data.js</code> , <code>StudyAbroad.css</code>

10.3 Recommended Engineering Standards (For All Devs & AI Agents)

- 1. **One data import path** — always `@/data/blogs` not mixed JSON/JS wrappers.
- 2. **Thin pages** — route files < 100 lines; logic in `src/features/` .
- 3. **No secrets in code** — fail fast if env missing.
- 4. **All POST APIs** — Zod validate + rate limit + consistent error JSON.
- 5. **Canonical URLs** — always `https://www.urbancode.in` .

- 6. **No new files in repo root** — scripts → scripts/ , data → src/data/ .
- 7. **Client components** — only when hooks/events needed; default to Server Components.
- 8. **CSS** — CSS modules or co-located; no 1000+ line global CSS files.
- 9. **PR checklist** — metadata, loading state, mobile check, no new hardcoded URLs.
- 10. **Delete dead code** — don't comment out 300-line components; remove or feature-flag.

10.4 Prioritized Refactoring List

Priority	Refactor	Files
P0	Remove credential fallbacks	lib/*Db.js, auth routes
P0	Add API auth middleware	app/api/feedback/*
P1	Split coursesData.js	Category JSON files
P1	Dedupe StudyAbroad.css	Remove ~50% duplicate blocks
P1	Delete temp_data.js, _projects/ clone	Root cleanup
P2	Extract email service layer	app/api/send-email/*
P2	Move compiler admin auth server-side	Compiler backend
P2	Replace HashRouter in compiler	Next nested routes or single page states
P3	Remove dead deps	package.json
P3	Consolidate FAQ components	Delete Faqs copy.jsx

11. Final Master Task List & Roadmap

11.1 Top 30 Tasks (Prioritized)

#	ID	Category	Task	Priority	Complexity	Impact	
1	T01	Security	Rotate MongoDB credentials; remove URI fallbacks from all files	P0	S	Critical	lib/feed lib/goog scripts
2	T02	Security	Implement JWT verification on all feedback admin API routes	P0	M	Critical	app/api/
3	T03	Security	Remove hardcoded admin/JWT/cron defaults; require env vars	P0	S	Critical	app/api/ sync rou
4	T04	Security	Move compiler admin auth server-side; remove client password	P0	M	High	AdminDas backend
5	T05	Security	Add rate limiting to /api/send-email/* and /api/chat	P0	M	High	API rout

6	T06	Bug	Fix <code>/form</code> to check <code>response.ok</code> before success UI	P0	S	High	<code>app/form</code>
7	T07	Security	Restrict <code>brochureUr1</code> fetch to allowlisted paths (fix SSRF)	P1	M	High	<code>send-cur</code>
8	T08	Security	Escape all user input in HTML email templates	P1	M	Medium	<code>app/api/</code>
9	T09	DevOps	Add GitHub Actions: lint + build on every PR	P1	M	High	<code>.github/</code>
10	T10	Testing	Smoke tests for course enquiry and contact form APIs	P1	M	High	<code>tests/in</code>
11	T11	Architecture	Add <code>middleware.ts</code> — <code>www</code> redirect + security headers	P1	M	High	<code>middlewa</code> <code>next.con</code>
12	T12	Bug	Fix category <code>deslugify()</code> fallback — use <code>notFound()</code> for unknown slugs	P1	S	Medium	<code>Courses.</code>
13	T13	UX	Remove or skip 3s Study Abroad nav animation delay	P1	S	Medium	<code>Navbar.j</code>
14	T14	Performance	Enable image optimization; convert large JPGs to WebP	P1	L	High	<code>next.con</code>
15	T15	Performance	Lazy-load navbar widgets (chatbot, book demo, 1-on-1)	P1	M	Medium	<code>Navbar.j</code>
16	T16	Performance	Shared cache for <code>/api/courses</code> across forms	P1	M	Medium	<code>EnquiryF</code>
17	T17	SEO	Unify sitemap — include blogs, study-abroad, all courses	P1	M	High	<code>app/site</code> <code>sitemap.</code>
18	T18	SEO	Add <code>generateMetadata</code> to blogs listing, study-abroad, course categories	P1	M	High	Multiple
19	T19	Architecture	Add <code>app/not-found.js</code> , <code>error.js</code> , <code>loading.js</code>	P1	S	Medium	<code>app/</code>
20	T20	Architecture	Delete <code>temp_data.js</code> , <code>keys.txt</code> , root migration scripts from repo	P1	S	Medium	Root file

21	T21	Architecture	Remove app/_projects/UC_Portfolio- main/ duplicate	P2	M	Medium	app/_pro
22	T22	Architecture	Split coursesData.js into per-category JSON	P2	L	High	coursesD src/data
23	T23	Performance	Compress/offload 70MB+ videos to Cloudinary	P2	L	High	public/v
24	T24	DevOps	Add /api/health + Sentry error tracking	P2	M	Medium	New rou
25	T25	Bug	Fix React Hooks violation in study-abroad page	P2	M	Medium	study-ab
26	T26	Bug	Add notFound() for invalid training slugs	P2	S	Low	training
27	T27	API	Paginate GET feedback responses/questions	P2	M	Medium	Feedbac
28	T28	Technical Debt	Dedupe StudyAbroad.css (remove repeated blocks)	P2	L	Medium	StudyAbr
29	T29	Feature	Admin CMS for blogs (stop hand-editing JSON)	P3	XL	High	New fea
30	T30	Feature	Lead status dashboard connected to CRM	P3	XL	High	New adr

11.2 Implementation Roadmap

PHASE 1 – CRITICAL SECURITY (Week 1-2)

- └─ T01-T05: Credentials, auth, rate limits
- └─ Deploy + verify + rotate secrets

PHASE 2 – STABILITY & BUGS (Week 3-4)

- └─ T06, T12, T13, T25, T26
- └─ T19: Error boundaries
- └─ T10: Smoke tests

PHASE 3 – ARCHITECTURE CLEANUP (Week 5-8)

- └─ T20, T21, T22, T28
- └─ T11: Middleware
- └─ Service layer extraction (email, feedback)

PHASE 4 – PERFORMANCE (Week 9-10)

- └─ T14, T15, T16, T23
- └─ Lighthouse baseline + targets

PHASE 5 – SEO & PRODUCTION HARDENING (Week 11-12)

- └─ T17, T18

- └ T09, T24
- └ Production readiness checklist sign-off

PHASE 6 – UX & PRODUCT (Week 13+)

- └ CMS for content
- └ Lead dashboard
- └ AI chatbot → enquiry bridge
- └ Student portal (long-term)

11.3 If Another Team Takes Over Tomorrow

They will immediately say:

"Why are database passwords in the Git repo?"
"Why can I DELETE all feedback with curl and no token?"
"Why is there zero tests and no CI?"
"Why is one file 10,000 lines?"
"Why do we have two portfolio apps and two feedback backends?"

Fix before calling it production-level:

1. Security emergency remediation (T01–T05)
2. Auth on admin APIs (T02)
3. CI + smoke tests (T09–T10)
4. Remove secret fallbacks and rotate credentials
5. Fix lead-capture false-success bug (T06)
6. Canonical URL + sitemap (T11, T17)
7. Delete duplicate/dead code (T20–T21)

12. Appendix — Health Scores

Dimension	Score	Rationale
Functionality	68	Core marketing, forms, blogs, courses work; admin/compiler gaps
Security	22	Hardcoded secrets, broken auth, no rate limits
Performance	45	Unoptimized images, huge assets, large bundles
Architecture	38	Monolithic data files, dual backends, no service layer
UX	58	Polished visuals; friction on nav, forms, mobile widgets
Testing	5	No automated tests
DevOps	35	Vercel deploy works; no CI, monitoring, or health checks
Scalability	40	Will strain on feedback/API abuse before traffic scale
Maintainability	32	AI-generated drift, duplicates, god files
Overall Production Readiness	42	Functional marketing site; not enterprise production-ready

Appendix B — Key File Reference

Purpose	Path
Root config	next.config.mjs, vercel.json, package.json
Main layout	app/layout.js
Course data (10k lines)	app/courses/[categorySlug]/coursesData.js
Blog data	lib/data/blogsData.json
RSS feed	app/rss.xml/route.js
Feedback DB	lib/feedbackDb.js
Email APIs	app/api/send-email/*
CRM integration	lib/api/externalEnrollment.js
Compiler backend	compiler/compiler/backend/server.js
Enquiry forms	app/components/common/EnquiryFormModal.jsx

Appendix C — How to Use This Document

1. **Share with leadership** — Executive Summary + Section 12 scores.
2. **Sprint planning** — Section 11 Top 30 tasks → Jira/ClickUp tickets.
3. **Security sprint** — Section 6 only; treat as immediate.
4. **AI agent rules** — Section 10.3 paste into `AGENTS.md` or Cursor rules.
5. **Re-audit** — After Phase 1–2, re-run audits and update scores.

End of Report — UrbanCode Edutech Production Audit, 27 August 2026
This document was generated from static codebase analysis. Runtime penetration testing and load testing were not performed.